

SISTEMAS DISTRIBUIDOS

CLASE 3 – REPLICACIÓN Y SHARDING



Hoja de ruta

1. ¿Por qué un sistema de storage no se puede tratar como un cómputo?
2. ¿Cómo se parte un dataset? Sharding
3. ¿De qué falla nos protege replicar?
4. ¿Cómo se replica?
5. ¿Qué es un log, y por qué lo resuelve todo?
6. ¿Quién decide el orden?
7. Chain replication
8. ¿Quién reconfigura el sistema?

STORAGE: SISTEMAS HECHOS DE PURO ESTADO

Por qué distribuir el storage

- Un sistema de storage en una única máquina siempre va a ser más simple
- Si de todos modos lo distribuimos, es por dos motivos:
- **Escalabilidad**, y en particular la horizontal: agregar máquinas y que la capacidad del sistema aumente
- **Tolerancia a fallas**, que acá se abre en dos:
 - alta disponibilidad: que el sistema no deje de responder, o que se restaure automáticamente
 - que no se pierda ningún dato, que es lo específico de un sistema de almacenamiento

Repaso: cómo lo resolvía MapReduce

- Escalabilidad: **particionar el problema** y mandar cada parte a una CPU distinta
- Tolerancia a fallas: la falla es **parcial**, y la partición que falló **se reintenta**
- Exige que map y reduce sean **deterministas**
- Deterministas $\Rightarrow$ **stateless**: con el input y el algoritmo alcanza para determinar el output

UN SISTEMA DE STORAGE ES PURO ESTADO

Los mappers y los reducers tenían que ser stateless. Un file system o una base de datos son exactamente lo contrario: **materializan la abstracción de memoria** dentro de un sistema distribuido

Distribución del Storage

- Dos técnicas para resolverla:
 - **sharding (particionado)**
 - **replicación**

SHARDING (PARTICIONADO)

Partir el dataset

- ¿Con qué criterio se parte?
- Cualquiera sea el criterio, hace falta mapear **cada dato a su ubicación física**
- Ese mapeo se puede pensar como una **función** (o una **tabla**)
- **Función:** el hash consistente, o Dynamo, que hasha la clave y con eso decide la partición
- **Tabla:** Google File System guarda en qué máquinas vive cada fragmento de 64 MB de cada archivo

Beneficios del sharding

1. **Escalabilidad** – (horizontal)
2. **Fallas parciales** – si falla una partición, el resto del sistema sigue funcionando
3. **Paralelismo de acceso** – se accede a varias particiones en simultáneo ->

Performance

Fallas parciales

- Si falla una partición, los clientes que no la necesitan **no lo perciben**
- Eso vale mientras no haya joins que crucen shards, algo que de por sí ya es complicado en estos sistemas
- Los clientes que sí la necesitan ven una baja de disponibilidad: el sistema está **degradado**, no caído
- Es un subproducto del sharding: la técnica por excelencia para tolerar fallas sigue siendo la replicación

Paralelismo de acceso

- Un cliente que necesita muchos datos de muchos lugares puede pedirlos a varias particiones **al mismo tiempo**
- Sube el **throughput** del sistema: más datos por segundo, porque el acceso se paralelizó
- Es exactamente lo que hace MapReduce en combinación con Google File System (próxima clase)

REPLICACIÓN

Copias del mismo dato

- En sharding cada máquina tiene datos **distintos**; en replicación las máquinas tienen **el mismo** dato
- El beneficio inmediato: si una muere, quedan otras a las que ir a buscarlo
- Y habilita una estrategia importante: poner las réplicas en **data centers distintos** -> protección contra desastres naturales

Paréntesis: RAID

- Replicar no siempre es copiar. Quien conozca RAID —*redundant array of inexpensive disks*— vio las dos variantes
- El modo **mirror** copia el dato en otro disco, tal como lo estamos describiendo
- Los otros modos usan **códigos de recuperación de errores**, códigos de Hamming, donde no se guarda una copia del dato
- La diferencia está en el espacio: con copias literales, un terabyte replicado tres veces cuesta **3 TB**, un sobre costo del **200 %**
- Con esos códigos, cinco máquinas guardan el equivalente a tres: **67 %**
- En esta materia siempre vamos a replicar copias literales

TIPOS DE FALLA

Fail-stop

- Mandamos un request y no obtenemos respuesta. No tenemos manera de saber qué pasó:
 - la máquina murió: se quemó el disco, se cortó la luz
 - o se rompió la red, y la máquina está viva del otro lado de la partición
- **Desde afuera son indistinguibles**, y por eso van en la misma categoría
- El sistema no emite respuestas incorrectas o corruptas

Fallas bizantinas

- **No podemos confiar en que la otra máquina esté haciendo lo que el protocolo dice**
- Una máquina que no respeta el protocolo: un bug que guarda $x = 2$ donde debía ir $x = 1$
- Una máquina maliciosa que busca engañarnos
- Se tratan con otro tipo de técnicas, y les dedicamos una clase aparte

¿CÓMO SE REPLICA?

State transfer (snapshot)

- La forma más intuitiva posible
- Tomar todo el estado del nodo 1 y transferírselo al nodo 2
- Problemas:
 - Lento
 - Hay que parar el mundo para evitar snapshots inconsistentes
- Pro: es simple
- Se suele combinar con el segundo método

Máquina de estados replicada

- Si una base de datos es determinista y le mandamos $o_1, o_2, o_3, o_4...$
- ...y a otra base de datos le mandamos exactamente esas mismas operaciones, en ese mismo orden...
- ...las dos terminan en el mismo estado
- Es una de las ideas que formalizó Lamport, y es la forma de replicar que domina todo el resto del curso

Requisitos para RSM

1. Las operaciones llegan en el **mismo orden**
2. **Determinismo estricto**: dado un estado y una operación conocida, el estado siguiente es uno solo
3. Todas parten del **mismo estado inicial**, S_0

GARANTIZAR EL ORDEN ES UN PROBLEMA DE CONSENSO

Problema: determinar **el orden de todas** las operaciones de manera global a medida que van apareciendo

EL LOG

El log

- Una estructura —concreta o conceptual— cuyos habitantes son **operaciones**
- No es el log de texto que un programa emite para debuguear. Es mucho más parecido al log de una base de datos
- Relaciona dos cosas que hasta ahora veníamos tratando por separado: **las operaciones y el orden**
- **Append-only:** se agrega siempre al final. Insertar en el medio invalidaría el orden ya establecido
- Y el orden es **total:** para todo par de operaciones se sabe cuál ocurrió antes

SI CONSEGUIMOS CORRECTITUD EN EL LOG DE OPERACIONES, RESOLVIMOS LA REPLICACIÓN

El log dice en qué orden ocurrieron los updates. Con eso, la máquina de estados replicada hace el resto

Ejemplo 1: el log de una base de datos

- Postgres tiene muchas estructuras adentro, pero dos son las grandes
- **Páginas:** donde se guardan las filas, organizadas en un árbol B+ con las hojas enlazadas
- **Write Ahead Log:** Antes "redo log"
- No es distribuido: es una máquina física con las páginas y el log adentro

El truco del WAL

- Se marca el **inicio** de la transacción, después las operaciones, y al final el **commit**
- Si la base se cae a mitad de una transacción, al arrancar —antes de aceptar requests— revisa qué transacciones iniciaron y no commitearon
- Cada entrada guarda el estado **inicial** y el **final** de la página que toca
- Con eso se revierte leyendo el log de atrás para adelante: sirve para la recuperación y también para el rollback explícito

Ejemplo 2: el log en Raft

- Dos capas en la misma máquina, y normalmente en el mismo proceso
- Arriba, **nuestra aplicación** con su storage: eso es lo que queremos replicar
- Abajo, **Raft**, con un log parecido al de la base de datos (pero sin transacciones)
- Cuando llega una escritura, Raft la agrega al log, la sincroniza con las otras máquinas, y **recién entonces** le avisa a la aplicación que la aplique

El log como dual del estado

- Cada operación tiene una **posición** en el log: 0, 1, 2, 3.
- Como la máquina es determinista y todas parten de S_0 , esa posición **identifica el estado** de la réplica
- Una réplica que aplicó hasta la 11 y otra que aplicó hasta la 10: sabemos cuál está atrasada
- El log termina funcionando como un **versionado** del estado de las réplicas

**¿CÓMO SE CONSIGUE EL
ORDEN?**

Idea 1: Active-Active

- El log afuera de las máquinas: la respuesta más barata es que lo mantenga otro
- Todas las escrituras del cliente van a ese log, y las réplicas **consumen** de él
- Son todas iguales entre sí y reciben las mismas operaciones en el mismo orden
- Un sistema que gestiona ese tipo de log es **Kafka**, cuyo paper vamos a leer más adelante
- Pero es el huevo y la gallina: no lo resolvimos, se lo delegamos a una caja negra que también tiene que mantener ese log

Idea 2: Primary-backup

- Una máquina designada como **primary** y varias otras que son sus **backups**
- Las escrituras van necesariamente al primary. Las lecturas pueden ir a los backups o al primary
- El primary actualiza su propio estado y después les manda las operaciones a los demás
- Otros nombres para lo mismo, según el contexto: master y slaves, writer y read-replica
- Casi todos los sistemas que veamos de acá en adelante son variaciones de este esquema

En la práctica: Postgres

- La forma habitual de escalar una base Postgres es exactamente esta
- Una instancia se define como el **writer**, de lectura y escritura; las otras quedan de solo lectura
- Lo que el writer les emite es su **WAL**: el mismo log de las transacciones, ahora usado para replicar
- Las réplicas se conectan a un slot de replicación y se van poniendo al día

EL PRIMARY DETERMINA EL ORDEN UNILATERALMENTE

No les pregunta a los backups si están atrasados. Determina el orden, **aplica localmente** y después **emite a las réplicas**. El consenso deja de ser *distribuido*

El precio: consistencia eventual

- El primary aplica, responde y emite. Las réplicas quedan levemente atrasadas
- Quien lee del primary obtiene la fuente de verdad; quien lee de un backup puede obtener una versión desactualizada
- Es el primer sistema **no linealizable** que vemos
- Linealizable, o de consistencia fuerte: si el sistema confirmó una escritura, **ninguna lectura posterior** ve la versión anterior, la haga quien la haga

El precio: pérdida potencial de datos

- Si falla un **backup** no es grave: se levanta uno nuevo y se lo pone al día
- Ponerlo al día es **snapshot + WAL**: una foto del primary y después el log hasta alcanzarlo
- Si falla el **primary**, puede haber aplicado una operación y muerto antes de emitirla
- Por eso pérdida *potencial*: puede coordinar, y esperar el acuse de algunos backups antes de responder

Ejemplo: Change Data Capture

- Las dos formas no son excluyentes, y la combinación es lo que efectivamente se encuentra montado en los sistemas reales
- Postgres emite su WAL, pero no a una réplica: a un nodo cuyo único trabajo es leerlo y publicarlo en un log
- Ese nodo existe y es open source: **Debezium**. Se conecta a la base simulando ser una réplica
- El log puede ser Kafka, y del otro lado hay consumidores que **no son** bases Postgres

El log es el dual de la base de datos

- Todos los consumidores reciben las mismas operaciones en el mismo orden:
todos son máquinas de estado replicadas
- Postgres define ese orden unilateralmente, y el resto lo sigue
- No son copias: son **vistas distintas de la misma información**, materializadas en tecnologías distintas
- De ahí la frase del artículo de Kreps: con el log se puede reconstruir la base

CHAIN REPLICATION

La cadena

- Tres nodos conectados en fila. Al primero lo llamamos **head** y al último **tail**
- El cliente escribe siempre al head, y la escritura se propaga eslabón por eslabón
- El que le responde no es el head: el **acuse vuelve del tail**
- Un RPC común no alcanza, porque el request va por un lado y la respuesta viene por otro: el cliente mantiene **dos conexiones** abiertas
- La escritura está segura recién cuando llega ese accuse

Los dos beneficios

- **Menos tráfico por máquina.** El primary tenía que mandarle cada escritura a los tres backups: tres copias por la misma placa de red
- Acá cada eslabón emite **una sola copia**, así que el costo de red por máquina no crece con la longitud de la cadena
- **Consistencia fuerte.** Leyendo siempre del tail, el sistema es linealizable
- El motivo está en el acuse: si el sistema confirmó, el dato ya recorrió toda la cadena y está en todos lados

Qué pasa si se lee del medio

- Un nodo intermedio puede tener un dato que todavía **no está fijo**
- El caso extremo: el head responde una lectura y muere antes de propagarla. Ese dato lo vio un solo cliente
- Y el cliente que la escribió nunca recibió el acuse, así que reintenta: quedó un dato que el sistema nunca confirmó
- Se puede relajar y leer de cualquier nodo, si el lector acepta datos desactualizados
- Lo que el tail garantiza, y los del medio no, es que el dato ya está en todos lados

Falla el head

- Supuesto por ahora: alguien externo a la cadena detecta la falla y reconfigura.
- Se promueve el segundo nodo a head, y hay que **reconfigurar a los clientes**, que tenían la conexión abierta con el anterior
- Si el head murió antes de replicar, el cliente nunca recibió el acuse: lo interpreta como fail-stop y reintenta
- Si llegó a replicar, la escritura continúa su recorrido y el tail responde de todos modos

Falla el tail

- El nodo anterior al tail pasa a ser el nuevo tail, y otra vez hay que reconfigurar a los clientes
- Si el sistema ya había respondido antes de la falla, el dato está garantizado en los otros nodos
- Si el tail falló antes de responder, el cliente reintenta y a lo sumo queda un **duplicado**, que hay que eliminar
- No se pierde información

Falla un nodo del medio

- Reconfigurar la cadena para **omitir ese nodo**
- Pero no está garantizado que todo lo que mandó el head haya llegado al tail
- Así que el head tiene que preguntarle al tail **en qué versión está**, y reenviarle lo que falte
- Eso obliga al head a **recordar**: no puede mandar una operación y olvidarse de ella. Necesita su propia memoria de log
- Variación que se usa en la práctica: el tail responde dos veces, al cliente y al head, y ese aviso permite limpiar

Agregar un nodo

- Se agrega **al final**, detrás del tail actual, y pasa a ser la nueva cola
- Llega vacío, así que primero recibe un **snapshot** y después el **log** de todo lo ocurrido desde esa foto
- Cuando alcanza la versión del que lo precede, se lo promueve a tail y **recién ahí** empieza a responder acuses
- Importa tanto como los tres casos anteriores: después de cada falla quedamos con un nodo menos, y hay que reponerlo
- Contado en una línea parece fácil. Coordinarlo sin perder ni duplicar datos es de las sutilezas difíciles

¿QUIÉN RECONFIGURA?

El split-brain

- El problema: las **particiones de red**
- Media cadena de un lado, media del otro, y clientes contra las dos mitades
- El nodo anterior al corte no obtiene respuesta y, como es fail-stop, **no puede distinguir** una partición de dos máquinas muertas
- Si se reconfigura solo, se autopromueve a tail y sigue operando. Del otro lado ocurre lo mismo, en espejo

Irreconciliable

- Lo que era un sistema con un único estado replicado pasó a ser **dos sistemas** que evolucionan por separado
- Las operaciones de un lado nunca llegan al otro, y viceversa
- Casi nunca se puede reparar automáticamente: la estrategia es **evitarlo**

El servicio de configuración

- La cadena, sola, no puede decidir cómo reconfigurarse. La salida es un servicio **externo**
- Ese servicio va detectando el estado de cada nodo, decide la configuración nueva y **se la comunica** a todos
- Nadie se autopromueve
- Para que no sea otra vez el huevo y la gallina, ese servicio tiene que soportar particiones. Hay dos formas de conseguirlo

Idea 1: una máquina sola

- Parece ingenuo, y sin embargo es lo que termina haciendo **Google File System**
- Ante una partición, esa máquina queda de un lado, y no hay alternativa posible
- Del lado donde quedó, ve que las máquinas del otro lado dejaron de funcionar: reconfigura la mitad con la que puede comunicarse, y esa mitad **sigue operando**
- La otra mitad recibe escrituras que nunca se confirman, porque nadie puede promoverle un tail. Queda **indisponible** hasta que vuelva la conexión
- Es drástico, pero **no hubo split-brain**. Y esta es la forma estándar de evitarlo: una mitad avanza, la otra queda no disponible

Idea 2: Raft (o Paxos, etc.)

- El servicio de configuración implementado como **tres o cinco nodos**, en vez de uno
- Con cinco y una partición de tres contra dos, avanza **la mayoría**: los tres
- Queda tolerante a fallas y a particiones, y después pone al día a los que quedaron atrás

Sharding y replicación van juntos

- Casi nunca aparecen por separado
- La clase que viene: un file system distribuido que parte los archivos en fragmentos de 64 MB —ahí está el sharding— y replica cada fragmento en tres, cuatro o cinco lugares
- Es **Google File System**, y usa primary-backup y sharding al mismo tiempo
- Evita el split-brain con un coordinador centralizado: exactamente la forma ingenua que acabamos de ver

Síntesis

Debajo de todo lo que vimos hay **una sola pregunta**: quién decide el orden

El active-active (log afuera) lo resuelve dejando que decida Kafka (o similar). El primary-backup, dejando que decida el primary, sin consultarle a nadie. La cadena, dejando que decida el head y que el tail confirme.